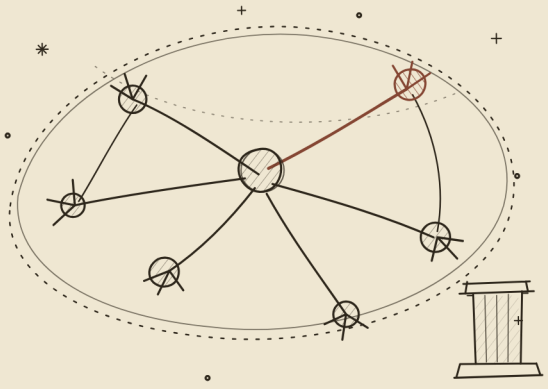
EXPEDIENTE · GUÍA DEL USUARIO

Proyecto Alejandría

Tu segundo cerebro



cada concepto, una neurona; cada relación, una sinapsis

1. Qué es y para qué sirve

Proyecto Alejandría es una aplicación de escritorio para Windows que convierte lo que aprendes en un **universo en 3D.** Cada concepto (la derivada, la distribución de Poisson, el movimiento browniano…) es una **neurona:** guarda su teoría, un glosario, ejercicios resueltos, referencias y un **laboratorio** donde puedes mover parámetros y ver cómo cambia el modelo.

Las neuronas se conectan entre sí mediante **sinapsis.** Las propone una inteligencia artificial que corre **en tu propio ordenador** (Ollama), explica por qué cree que dos conceptos se relacionan y te deja aceptar o rechazar cada una. Nada sale de tu equipo: no hay cuentas ni envío de datos, y la nube solo entra si tú eliges una carpeta para sincronizar tus PCs.

Sirve para estudiar y repasar, para construir un mapa propio de una materia y para entender un modelo tocándolo, no solo leyéndolo. Es un proyecto personal de código abierto (licencia MIT) y por ahora solo funciona en Windows 10 y 11.

Pensado para que tu conocimiento sea tuyo: en una carpeta local, exportable y sin depender de nadie.

2. De un vistazo

Neuronas	Teoría con fórmulas, glosario, ejercicios con solución, referencias y laboratorio.
Sinapsis	Conexiones propuestas por una IA local, con su razón; tú decides.
Laboratorios	Simulaciones en Python que se ejecutan dentro de la ventana, sin internet.
Varios PCs	Una carpeta de tu nube (OneDrive, Dropbox…) mantiene tus neuronas iguales en dos ordenadores.
Actualizaciones	La app avisa de las versiones nuevas y se actualiza sola cuando tú lo pides.
Privacidad	Todo en local; el servidor solo escucha en tu equipo.

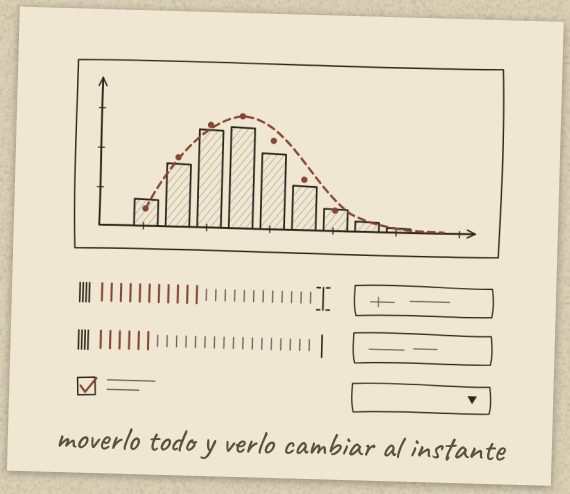
Cómo se usa

paso a paso

3. Funcionalidades y mecánicas

Crear una neurona

1. Pulsa `[ + Inyectar neurona ]` (abajo a la izquierda).
2. Copia uno de los dos prompts: *Fuentes (doctorado)* o *Exploración libre*, este último pide a la IA consultar también Google Scholar.
3. Pégalo en tu IA favorita con el tema que quieres aprender y pega aquí el JSON que te devuelva.
4. Pulsa `[ Asimilar neurona ]`: se guarda, se clasifica en una rama y la IA local busca sus conexiones (verás el pilar girando y una barra de avance).



moverlo todo y verlo cambiar al instante

Explorar el universo

Arrastra para girar, rueda para acercarte y haz clic en una neurona para abrirla. Si te acercas mucho, los nombres de las neuronas que tienes delante aparecen tenues. La barra superior busca por nombre, rama o glosario (flechas y Enter para elegir); `[ Centrar vista ]` vuelve a la vista completa.

El panel de una neurona

Seis pestañas: **Teoría**, **Glosario**, **Laboratorio**, **Ejercicios** (con la solución oculta hasta que la pidas), **Referencias** y **Sinapsis**. Desde la cabecera puedes corregir el clúster a mano o destruir la neurona.

Sinapsis

Cada conexión nueva queda pendiente con la razón que dio la IA, que escribe en castellano. `[ Aceptar ]` la consolida; `[ Rechazar ]` la elimina y **no vuelve a proponerse**. También puedes crear una conexión manual con tu propio motivo. Si Ollama no está en marcha, la app lo arranca sola en segundo plano.

Laboratorios

Mueve las barras `[#####-----]` (con el ratón o las flechas del teclado) y el gráfico se redibuja al momento: barras, líneas, áreas, escala logarítmica… Con `[ Editar laboratorio ]` añades o quitas variables, cambias rangos y editas el código Python. Un laboratorio que viene de un JSON pegado pide tu aprobación antes de ejecutarse: léelo antes de aceptar.

Ajustes

Activa o desactiva la IA local, elige el modelo, **exporta e importa** tus neuronas en un archivo, carga tres neuronas de ejemplo, repite el tutorial y consulta las licencias de terceros. Desde la 0.2 también controlas ahí las **actualizaciones** y la **sincronización entre PCs**. Importar nunca pisa una neurona que ya tengas.

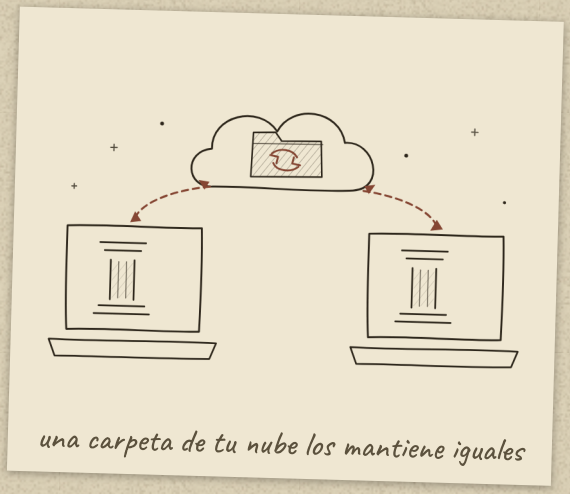
Si desactivas la IA, las conexiones se proponen solo por parecido de textos.

Varios PCs, al día

desde la 0.2

4. Varios ordenadores

Si usas la app en dos PCs, en **Ajustes → Sincronización entre PCs** eliges en los dos una carpeta que ya sincronice tu nube (OneDrive, Dropbox, Google Drive, Syncthing…). Cada PC deja ahí lo suyo, en su propia subcarpeta, y lee lo de los demás: **sin servidor ni cuentas nuevas.**



una carpeta de tu nube los mantiene iguales

- Viajan las **ediciones**, las **neuronas nuevas**, los **borrados** y qué conexiones aceptaste o rechazaste. No viajan los ajustes ni los cálculos internos de cada PC.
- Se sincroniza **al abrir la app** y cuando pulsas `[ Sincronizar ahora ]`; no hay nada corriendo todo el rato.
- Si editas lo mismo en dos PCs a la vez, gana lo más reciente y lo otro se guarda aparte, en la carpeta `conflictos` de tus datos.

> **Cuidado con los datos.** Si la carpeta no está disponible no se cambia nada; un archivo a medio subir se ignora y se reintenta; y si de golpe desaparecen muchas neuronas locales, no se tratan como borrados sino que se restauran. Los datos pasan por la nube que tú elijas: la app no sube nada por su cuenta.

5. Actualizaciones

Al abrirse, la app pregunta a github.com si hay una versión nueva (no envía nada tuyo; se puede apagar en Ajustes). Si la hay, aparece un **cartel pequeño** en la esquina de abajo a la derecha: la versión, un «qué hay de nuevo» plegado y dos botones, `[ Luego ]` y `[ Actualizar ]`.

1. `[ Luego ]` oculta el cartel hasta la próxima vez que abras la app. No se instala nada si no pulsas `[ Actualizar ]`.
2. Al pulsar `[ Actualizar ]` se descarga el instalador con una barra de avance y la app comprueba su **firma digital** con la clave pública que lleva dentro: solo se instala lo firmado por el proyecto.
3. La app se cierra, una ventanita mínima la actualiza (no se abre el asistente completo) y vuelve a abrirse con un aviso de 5 segundos: «Actualizado a la versión X». Tus neuronas no se tocan.

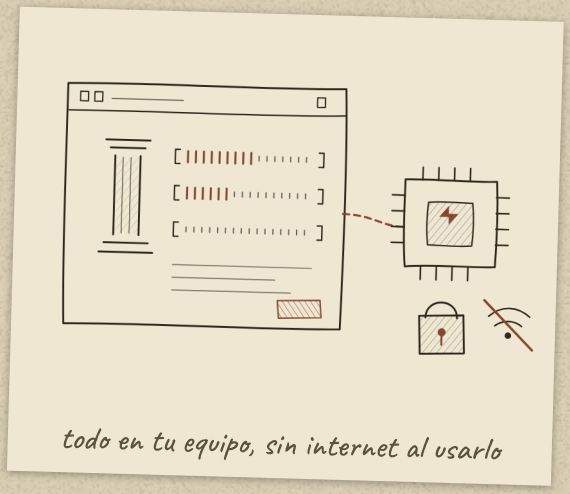
Las versiones anteriores a la 0.2.0 se instalan a mano una vez; a partir de ahí, ya se actualizan solas.

Instalar y por dentro

Windows 10 / 11

6. Instalación y requisitos

- **Sistema:** Windows 10 u 11 de 64 bits, sin ser administrador, con el *Visual C++ Redistributable* actual (14.40 o posterior); el instalador lo comprueba.
- **Descarga:** el instalador `Proyecto-Alejandria-Setup.exe` está en la página de releases del repositorio. Opcional: verifica el hash con `SHA256SUMS.txt`.
- **Espacio:** el «motor» (Python y librerías) ocupa unos 500 MB, el modelo de clasificación ~240 MB y Ollama, si lo eliges, 1,6 GB más el modelo.
- **Internet:** solo para instalar (se baja todo una vez), buscar actualizaciones y sincronizar por tu nube.



todo en tu equipo, sin internet al usarlo

Aviso de Windows. El instalador no está firmado (la firma de código cuesta dinero y es un proyecto personal), así que SmartScreen mostrará «Windows protegió su PC». Pulsa Más información y luego Ejecutar de todas formas, solo si lo has descargado de la página oficial del proyecto.

El asistente te deja elegir dónde van la aplicación, el motor y tus neuronas, y si instalas Ollama y con qué modelo. Con *Examinar* cada parte va a su subcarpeta (`Aplicacion`, `Motor`, `Neuronas`, `Modelos`), o marca «Instalarlo todo en una sola carpeta». Al terminar abre la app con un tutorial. Si algo falla, mira `instalacion.log` y `backend.log` en la carpeta del motor; vuelve a ejecutarlo y retoma lo que falte.

7. Detalles técnicos

Interfaz	Tauri 2 (Rust) + React + Three.js; instalador NSIS propio.
Servidor local	FastAPI en `127.0.0.1`, datos en Parquet, embeddings con fastembed.
IA	Ollama (`qwen2.5:1.5b`, `phi3:mini` …) para juzgar sinapsis; opcional.
Laboratorios	Pyodide (Python en WebAssembly: numpy, scipy, matplotlib, pandas) aislado en un worker; CSP estricta.
Actualizador	Plugin de actualización de Tauri con firma minisign (Ed25519).
Sincronización	Cada PC escribe solo en su subcarpeta; conflictos guardados aparte.

Ejecutarlo desde el código (Python 3.11+, Node.js 20+ y Rust)

```
alejandria.bat
cd frontend && npm install && npm run tauri dev
```

Código MIT en `github.com/MarioGreks/proyecto-alejandria`. Tests: `unittest` y `npm test`.